

Appunti: Differenza tra queue e deque in C++

1. std::queue (Coda FIFO)

La **queue** implementa la logica **FIFO** (First-In, First-Out). Pensa alla fila alla cassa: il primo che arriva è il primo a essere servito. È una struttura molto restrittiva: non puoi scorrere gli elementi al suo interno né accedere a quelli centrali.

Metodi principali:

- **push(x)**: Inserisce l'elemento **x** in fondo alla coda.
- **pop()**: Rimuove il primo elemento (non restituisce il valore).
- **front()**: Legge il valore del primo elemento.
- **empty()**: Restituisce **true** se la coda è vuota.

```
#include <iostream>
#include <queue>
using namespace std;

int main() {
    queue<int> fila;
    fila.push(10); // La fila e': [10]
    fila.push(20); // La fila e': [10, 20]

    // Svuotiamo la coda per leggerla (non si puo' usare il for!)
    while(!fila.empty()) {
        cout << fila.front() << " "; // Legge il primo (10, poi 20)
        fila.pop();                  // Rimuove il primo
    }
    return 0;
}
```

2. std::deque (Double-Ended Queue)

La **deque** è una coda "a doppio fine". Immaginala come un **vector** potenziato: ti permette di aggiungere e rimuovere elementi in modo super efficiente sia all'inizio che alla fine. A differenza della **queue**, qui **puoi accedere agli elementi tramite indice** e usare i normali cicli **for**.

Metodi principali:

- **push_back(x)** / **pop_back()**: Aggiunge / Rimuove in fondo (come il vector).
- **push_front(x)** / **pop_front()**: Aggiunge / Rimuove in cima (veloce!).
- **operator[i]**: Accede all'i-esimo elemento, proprio come un array.

```
#include <iostream>
#include <deque>
using namespace std;

int main() {
    deque<int> mazzo;
    mazzo.push_back(10); // In fondo: [10]
    mazzo.push_back(20); // In fondo: [10, 20]
    mazzo.push_front(5); // In cima:  [5, 10, 20]

    mazzo.pop_back();    // Rimuove l'ultimo (il 20): [5, 10]

    // Posso usare un normalissimo ciclo for con gli indici!
    for(int i = 0; i < mazzo.size(); i++) {
        cout << mazzo[i] << " "; // Stampa: 5 10
    }
    return 0;
}
```